

DNDROM / RESEARCH

Believable trees and dense foliage

Species architecture / Continuous trunks / Multi-scale foliage

Prepared 6 September 2026 for DnDRom's local procedural-world implementation. Scope: tree architecture, trunk geometry, leaf shading, forest ecology, grass, distant representations, and validation in PlayCanvas 2.21.4.

The largest gains come from correct structure and preserved coverage. Increasing tree counts alone magnifies the current problems. Generate species-specific trees locally, share a small library of seeded prototypes, and replace expensive detail according to projected size while retaining the same forest boundaries.

Correct structure, preserved coverage

Tree structure, bark and leaf shading, forest ecology, irregular grass and horizon representations. Findings tied to the current implementation, with remaining work stated explicitly.

Evidence-backed recommendations and source audit. Performance targets and proposed features are clearly separated from validated implementation.

Tree structure and the trunk defect

The existing trunk was a chain of separately generated cones. Each end radius was clamped below 82% of its start radius, then the next segment restarted wider. This creates repeated collars resembling a palm trunk. UVs restarted over every short segment, and branch-frame axes were recalculated independently. This is a repository finding, not an inference from a paper.

The correction uses one radius per graph node, smoothly transported tube frames, shared rings along dominant branch chains, and bark UV distance measured along the branch. Root flare decays with height. Side branches begin inside the supporting limb with their own diameter. The current implementation overlaps lateral tubes; it does not construct a watertight manifold union at forks.

Tree architecture needs biological controls in addition to attraction points. Apical control, bud competition, branching order, phyllotaxis and environmental responses produce distinct growth habits. A species recipe should expose these controls alongside crown envelope and age. [Palubicki et al., Self-organizing Tree Models for Image Synthesis, ACM SIGGRAPH, 2009.](#)

Implementation choice: retain attraction-driven growth for irregular deciduous crowns, but give conifers a persistent central leader and lateral branch whorls. Approximate supporting cross-sectional area from downstream twig length, then derive radius from its square root. This is an engineering heuristic, not a calibrated biological growth simulation. Seeded height, spread, branch rotation and crown asymmetry provide coherent variation.

Leaves, bark and lighting

The audit found identical normals on every leaf, oversized diamond leaves on pine trees, and normal maps removed from bark by a shared vegetation-material function. Dark brown vertex colors also multiplied already-brown bark albedo. Correcting geometry without correcting these material paths leaves trunks dark and foliage flat.

Leaf and conifer branchlet cutouts now use locally generated textures. A broadleaf card represents several small leaves; an evergreen card represents a needle spray. Per-card normals follow orientation. The bark material retains its albedo and normal detail. Alpha mip levels compensate for coverage loss so thin sprays do not simply vanish as textures shrink. Alpha testing still needs visual evaluation under TAA, shadowing and oblique views.

Vegetation shading benefits from separate broad plant motion and leaf flutter, stiffness/phase variation, ambient occlusion and thickness-sensitive backlighting. A leaf should respond differently when lit through its surface than opaque bark. [Tiago Sousa, Vegetation Procedural Animation and Shading in Crysis, GPU Gems 3, NVIDIA, 2007.](#)

Recommended next shading step: store branch order, stiffness and leaf thickness in packed vertex channels, use coherent wind for trunk and shadows, and add restrained transmission. Keep bark rough and detailed. Do not compensate for temporal shimmer by removing every normal map. Alpha-to-coverage requires a compatible multisampled target; it is not a general replacement for TAA or alpha-tested coverage handling.

Dense forests and irregular grass

Use coherent ecological fields for canopy, understory and ground cover. Forests need overlapping crowns, small gaps, saplings, shrubs, leaf litter and biome-specific grass. Keep waterways, road shoulders, steep rock and building footprints authoritative exclusions. Density should depend on local moisture, exposure and light, rather than solely on a random threshold.

The current ground-cover LOD selected every second or fifth entry from a row-ordered array. This creates spatial bands. Keep the accepted locations and reduce blade geometry within each clump, compensating approximate projected area. Appearance randomness must be independent of acceptance priority: otherwise sparse areas systematically receive the same size, rotation and variant.

Bridson's Poisson-disk method uses an acceleration grid and an active list to produce evenly separated random samples in linear expected work for a fixed attempt count. It is a useful reference for removing grid patterns. [Robert Bridson, Fast Poisson Disk Sampling in Arbitrary Dimensions, SIGGRAPH sketch, 2007.](#)

DnDRom's current grass approach is a different, bounded method: world-anchored independent jitter plus local priority-based exclusion, including neighbors outside the chunk. It preserves deterministic boundaries without global sequential state. Validate minimum spacing and chunk-union identity; do not call this an exact implementation of Bridson's algorithm.

Natural surface texture tiling can be reduced using stochastic patch blending. Heitz and Neyret combine three samples on a triangular grid with histogram-preserving blending. Deliot and Heitz describe practical color-transform and mip/filtering considerations. Albedo, normal and roughness must use corresponding transforms. [Heitz and Neyret, High-Performance By-Example Noise using a Histogram-Preserving Blending Operator, 2018](#) and [Deliot and Heitz, Procedural Stochastic Textures by Tiling and Blending, 2019.](#)

Forests from ground level to the horizon

Millions of world assets should not mean millions of detailed meshes submitted every frame. Manage world-anchored spatial cells, share prototype geometry, bound generation/upload queues, and use complementary LOD transitions. Grass and trees need separate visibility ranges and independent representations. [David Whatley, Toward Photorealism in Virtual Botany, GPU Gems 2, NVIDIA, 2005.](#)

Representation	Preserve	Reduce
Close tree mesh	Trunk silhouette, connected limbs, individual leaf sprays	Prototype duplication; tiny hidden twigs
Intermediate mesh	Every major crown region and approximate coverage	Radial tube segments, sprays per crown region, shadow detail
Distant tree impostor	Species silhouette, apparent height, directional appearance	Full branch and leaf geometry
Horizon canopy	Forest boundaries, canopy height, skyline and aggregate shading	Individual tree identity in the raster representation

A forest aggregate must represent more than a green terrain channel. Bruneton and Neyret describe transitions between individual-tree representations and an aggregate forest representation with consistent lighting. This supports the representation hierarchy; their performance is not a benchmark for DnDRom. [Eric Bruneton and Fabrice Neyret, Real-Time Realistic Rendering and Lighting of Forests, Computer Graphics Forum, 2012, publisher abstract.](#)

The original atlas stopped trees above terrain LOD 2. The working tree now uses four-triangle, two-projection crown impostors at LOD 2 and extends atlas trees through terrain LOD 4. These are intermediate approximations; a continuous horizon canopy and robust transitions remain unfinished. Keep parent forest coverage visible until the replacement is ready. Select by projected size with hysteresis, not by deleting every nth branch or by arbitrary terrain-tile level alone.

PlayCanvas supports hardware instancing and indirect drawing. Shared geometry reduces draw submission overhead, but spatial culling, shadow budgets, fragment overdraw and texture bandwidth remain independent costs. Verify the installed 2.21.4 APIs because current online documentation may describe a newer release. [PlayCanvas, Hardware Instancing](#) and [PlayCanvas, Indirect Drawing](#).

Acceptance gates and implementation status

Implemented in the working tree: continuous trunk radii and swept bark chains; connected limb pruning; conifer-specific branching; seeded architectural variation; corrected foliage normals; procedural leaf and needle sprays; alpha coverage mip adjustment; preserved bark maps; row-free LOD selection; independent placement appearance; denser forest ecology settings. These are local changes, not a released installer.

Gate	Required evidence	Status / limitation
Tree structure	Deterministic connected graphs, continuous single-child radii, finite meshes at all LODs	Focused geometry tests pass
Close visual quality	Trunks and forks, all species, daylight and backlighting, bark and foliage close-ups	Native WebGPU review completed for one forest scene; broader species and lighting review pending
Forest density	New generation in several biomes, roads and water excluded	Settings changed; broad visual acceptance pending
Grass	No rows after generation, stable chunk edges, density preserved through LOD	Deterministic chunk-union and minimum-spacing tests pass; broad visual review pending
Distant forest	Smooth mesh/impostor/canopy transition, visible skyline, travel to same locations	Intermediate crossed projections implemented; aggregate canopy and smooth transitions pending
Performance	CPU and GPU frame times, p50/p95, cold streaming stalls, draw calls, shadows and memory on target hardware	Native sample remains around 23 FPS; optimization target not met

Benchmark at ground level, above canopy and from a mountain viewpoint. Traverse the same route in both directions to expose unstable IDs and LOD popping. Record cold generation separately from warm steady rendering. Compare quality at a fixed resolution and camera path; one FPS sample does not establish sustained performance.

Research is sufficient to choose the architecture. Remaining uncertainty concerns implementation quality, material calibration and measured budgets, not whether instancing and multi-scale representations exist. The source review included original tree-growth research, published rendering chapters, sampling and stochastic-texture work, and official engine documentation. The forest-lighting paper was available as an abstract; detailed numerical claims from its inaccessible full text were excluded.

Engine decision: favor an Unreal prototype

The product requirement is an explorable, highly detailed local fantasy world. With splats no longer the deciding reason to retain PlayCanvas, my recommendation is a bounded UE5 renderer prototype before investing further in a custom virtualized renderer. This is an engineering recommendation, not proof that migration is faster or that Unreal will meet the eventual minimum hardware target.

Nanite analyzes imported meshes into hierarchical triangle clusters, chooses visible detail according to the view, and streams data on demand. It addresses geometry submission and detail management at a deeper level than our current instancing. Practical limits still depend on content and hardware. [Epic Games, Nanite Virtualized Geometry, UE 5.8 documentation, accessed 6 September 2026.](#)

Epic's newer Nanite Foliage pipeline combines assemblies, voxels and skinning. Voxel representations target disconnected foliage and retain crown volume at distance. Epic currently labels this pipeline Experimental; evaluate it separately from established Nanite static-mesh support. [Epic Games, Nanite Foliage, UE 5.8 documentation, accessed 6 September 2026.](#)

The crucial compatibility gate is runtime-created geometry. Epic's current Geometry Scripting guide explicitly lists Nanite, mesh distance fields, LODs and instancing as unsupported on DynamicMeshComponent. The existence of runtime mesh generation therefore does not establish a runtime Nanite build pipeline. Prepared Nanite components, procedural assembly and arbitrary new mesh topology are separate paths that must be tested in a packaged application. [Epic Games, Geometry Scripting Users Guide, UE 5.8 documentation, accessed 6 September 2026.](#)

PCG runtime generation can generate and clean up content around generation sources in standalone builds. Hierarchical generation separates prominent assets from nearby detail. Its GPU processing path supports point processing and static-mesh spawning, but is documented as Beta. These systems can support procedural distribution; they do not themselves synthesize any requested architecture. [Epic Games, PCG Generation Modes](#) and [Epic Games, PCG with GPU Processing.](#)

Preserve campaign schemas, seeds, edits and the owned local model runtimes. Give UE responsibility for the world renderer, camera, collision and interaction. Extract runtime ownership behind a local process interface; Tauri-bound calls and renderer-specific shaders need adapters or replacements. The current machine has UEFN/Fortnite installed but no full Unreal Engine registration was found in the checked launcher locations. No Unreal prototype has been compiled, and no engine migration is claimed complete.

Reference images on an 8 GB / 16 GB machine

The product baseline is now 8 GB VRAM and 16 GB system RAM. The development machine was verified as an RTX 3080 with 10 GB VRAM, Ryzen 7 5800X and approximately 32 GB RAM. Success on that machine cannot establish success at the baseline. Graphics feature support and GPU throughput also matter; memory capacity alone is not a performance class.

Treat the image as evidence for composition, style, rough depth and visible objects. Build an editable semantic scene with terrain, building footprints, rooms, entrances, paths, plants and material roles. Geometry behind an occluder and unpictured interiors must be designed or inferred. A single-view resemblance does not establish a complete reconstructed world.

TripoSR is the first conservative optional prop benchmark: its authors document approximately 6 GB VRAM for default single-image reconstruction, with mesh output and optional colour-texture baking. It reconstructs isolated objects, not whole semantic scenes. Its advertised inference time uses an A100 and is not a target-PC timing. [VAST / Tripo AI / Stability AI, TripoSR, official repository, 2024](#).

Hunyuan3D-2 provides a second shape-quality candidate. Its official repository states 6 GB for shape generation and 16 GB for shape plus texture. The 2mini model is a 0.6B shape generator. Neither statement validates concurrent Unreal, vision, speech and generation within 8 GB VRAM or 16 GB host RAM. [Tencent Hunyuan3D Team, Hunyuan3D-2, official repository, 2025](#) and [Hunyuan3D-2mini official model card](#).

Recommended sequence: analyze the reference locally; release the vision model; construct a procedural scene; generate selected unique props one at a time; release the generation process; upload cached meshes and materials; validate collisions and navigation. During a heavy job, reduce the renderer's residency or show a lightweight scene preview. Unloading competing models is preferable to assuming unlimited CPU offload on a 16 GB system.

Use prepared Nanite components where they fit, plus conventional runtime meshes for novel topology until a packaged Nanite build path is proven. Cook a bounded family of material shaders and let local AI change texture maps and exposed parameters. Preserve the semantic recipe and original images so edits and regeneration remain possible. This does not require a finite list of named village styles; procedural construction and material parameters can be driven by open-ended descriptions.

The reviewed evidence does not establish a one-model, complete editable image-to-world solution at this hardware target. Low-memory claims for experimental forks and whole-world demos were excluded from the baseline. No AI model benchmark was run in this task. The first Unreal milestone must be a packaged Windows build that reconstructs one reference-guided forest settlement, preserves edits, and measures total CPU/GPU memory through generation and exploration.